

Setting up your own Manuscript Manager

A step-by-step guide for people who do not write software

What you are about to build

A private website that watches your Gmail inbox, recognises letters from journals, and keeps one up-to-date card for every paper you have submitted — from first submission, through rejection and resubmission elsewhere, to publication. When a journal gives you a deadline, it sends a WhatsApp reminder before the date passes.

It runs by itself, every hour, for free, with no server to rent and no monthly bill.

What it costs

Nothing. Every service used here has a permanently free tier that this app stays far inside. You will not be asked for a card at any point. If a page ever asks you to pay, you have taken a wrong turn — stop and re-read the step.

How long it takes

About **an hour and a half**, spread over eleven parts. You can stop after any part and come back; nothing breaks half-finished.

Part	What it does	Time
1	Put the app into your own GitHub account	20 min
2	Clear out the previous owner's data	5 min
3	Tell the app who you are	10 min
4	Let it read your email	25 min
5	Give it a free AI key	5 min
6	Put the keys into GitHub	10 min
7	Switch the website on	5 min
8	Run the first sync	10 min
9	Change the password	5 min
10	WhatsApp reminders	10 min
11	The Edit button (optional)	25 min

What you need before you start

- A **computer** (not a phone — some steps need a real keyboard and a file window).
- A **Gmail account** for every inbox you want watched.

- About **90 minutes**. Parts 4 and 11 are the fiddly ones; the rest is typing things into boxes.
- The **ZIP file** of the app, which the person who sent you this guide will give you.

You will **not** need to install any software, use a black terminal window, or type a single command. Everything happens in your web browser.

Three things to know before you begin

1. Your paper list will be public

The website is free because GitHub hosts it free, and GitHub only hosts websites free from **public** folders. Public means exactly that: anyone on the internet who knows the address can read your list of papers, which journals rejected them, and which are still under review.

The website asks for a password, but that password is a curtain, not a lock — it stops someone glancing at your screen, and it does not stop anyone who goes looking. **Assume everything the tracker knows is readable by anyone.**

If that is not acceptable for your group, you have two choices: pay for GitHub Pages on a private repository (about **\$4 per month** on GitHub Pro), or do not set this up. There is no free way to have both.

Decide this now, with your co-authors, rather than after six months of correspondence is in there.

2. The app can read all of your email

To find the twelve letters a month that come from journals, it has to look at everything that arrives. It only ever **reads** — it can never send, reply, delete or change anything, and the permission it asks Google for does not allow those things even by accident.

The subject line and sender of mail it decides to ignore are written into a log file, which on a public repository is also public. Personal mail that happens to arrive in the same inbox is therefore partly exposed. If you share an inbox with anything sensitive, use a different Gmail account for this.

3. There is a default password, and everyone has it

The app ships with the password `orgkarur2026`, which is printed in this guide and therefore known to every person who has ever read it. **Part 9 changes it.** Do not skip Part 9.

A few words you will meet

You do not need to understand these deeply — just enough to recognise them.

Word	What it means here
Repository (or <i>repo</i>)	A folder of files stored on GitHub. Yours will hold the app and its data.
Commit	A saved change to that folder. GitHub keeps every one, so nothing is ever truly lost.
Actions	GitHub's way of running a task on a schedule. This is what checks your email every hour.
Secret	A password or key you give GitHub to use but never show. Once saved, even you cannot read it back.
Token	A long password issued by a service to a program, instead of to a person.
Workflow	One scheduled task. This app has several; you will mostly use <i>Sync manuscript tracker</i> .

Part 1 — Put the app into your own GitHub account

Time: about 20 minutes.

Step 1.1 — Create a GitHub account

Skip this if you already have one.

1. Go to <https://github.com/signup>.
2. Enter your email, choose a password, choose a username. Your username becomes part of your website address, so pick something you are happy to show colleagues — `sarah-ortho-lab` rather than `xx_bones_Xx`.
3. Verify the email they send you.
4. When asked which plan, choose **Free**.

Step 1.2 — Unzip the file you were given

1. Find the ZIP file (usually in your **Downloads** folder).
2. **Windows:** right-click it → *Extract All* → *Extract*. **Mac:** double-click it.
3. You now have a folder. Open it. You should see folders named `assets`, `config`, `data`, `scripts`, `worker`, and files including `index.html`, `README.md` and `package.json`.

Make the hidden folder visible — do not skip this

One of the folders is called `.github`, and the dot at the front makes it invisible on a Mac. It is the single most important folder in the app: it contains the instructions that check your email every hour. If it does not reach GitHub, **nothing will ever run**, and the failure is silent.

- **On a Mac:** open the folder in Finder and press **Cmd + Shift + .** (command, shift, full stop). A folder called `.github` appears, greyed out. Leave it visible for now.
- **On Windows:** nothing to do. It is already visible.

Step 1.3 — Create your repository

1. Go to <https://github.com/new>.
2. **Repository name:** `Manuscript-Manager` (You may choose another name, but then you must use *your* name everywhere this guide says `Manuscript-Manager`. It is easier to keep it.)
3. Choose **Public**. (See warning 1 above. Private needs a paid plan for the website to work.)
4. Leave *Add a README file* **unticked** and both dropdowns on **None**. You are about to upload files, and a pre-made file gets in the way.
5. Click **Create repository**.

You land on a mostly empty page with some setup instructions. Ignore them.

Step 1.4 — Upload the app

1. On that page, find the link **uploading an existing file**. (If you cannot see it: click **Add file** → **Upload files**.)
2. Open your unzipped folder in a separate window and arrange it so you can see both windows.
3. Select **everything inside** the unzipped folder — including the greyed-out `.github` and `.gitignore`. Use **Ctrl + A** (Windows) or **Cmd + A** (Mac).

Select the *contents*, not the folder itself. If you drag the folder, all your files end up one level too deep and nothing works.

4. Drag the selection onto the GitHub upload area.
5. Wait. There are several hundred files and the counter climbs slowly. Do not close the tab.
6. When it settles, scroll to the bottom and click **Commit changes**.

Step 1.5 — Check it arrived properly

This check takes ten seconds and saves an hour.

Look at your repository's file list. You must be able to see a folder named `.github`.

- **If you can see it:** good, continue.
- **If you cannot:** the hidden folder did not upload. Go back to Step 1.2, make hidden files visible, then **Add file** → **Upload files** again and drag only the `.github` folder across. Commit.

Also confirm you can see: `assets`, `config`, `data`, `scripts`, `starter`, `tools`, `worker`, and `index.html`.

An easier alternative, if this went badly. Ask whoever sent you the ZIP to open their repository's **Settings** and tick **Template repository**. A green **Use this template** button then appears on their page, and clicking it gives you a perfect copy in your own account in one step, hidden folders and all. If they do that, you can replace all of Part 1 with that one click — but you must still do Part 2.

✓ Checkpoint

Your repository exists, and `.github` is in the file list.

Part 2 — Clear out the previous owner's data

Time: 5 minutes. Do not skip this part.

The ZIP you were given is a copy of somebody's working tracker, so it contains **their** papers — every title, journal, rejection and deadline. If you leave it, your dashboard will show a hundred papers that are not yours, and your first sync will try to fold your mail into their records.

There are five files to replace. Empty versions are already in the `starter` folder, so this is copy-and-paste, five times.

Step 2.1 — Replace the five data files

Do this once for each of the five files listed in the table below.

1. In your repository, click the `starter` folder, then `data`, then the first file, `manuscripts.json`.
2. You will see something very short, like `{ "generatedAt": null, "manuscripts": [] }`. Select all of that text and copy it (**Ctrl/Cmd + A**, then **Ctrl/Cmd + C**).
3. Go back to the top of your repository (click the repository name at the top of the page), click the `data` folder, then `manuscripts.json`.
4. Click the **pencil icon** () near the top right to edit it.
5. Select everything in the box (**Ctrl/Cmd + A**) and paste over it (**Ctrl/Cmd + V**).
6. Scroll down, click **Commit changes**, then **Commit changes** again in the box that appears.

Repeat for all five:

Replace this file in <code>data/</code>	With the one of the same name in <code>starter/data/</code>
<code>manuscripts.json</code>	your list of papers — the big one
<code>sync-state.json</code>	the record of which emails have been read
<code>excluded-log.json</code>	the log of mail that was read and set aside
<code>review-queue.json</code>	mail the AI was unsure about
<code>notifications.json</code>	which reminders have already been sent

Step 2.2 — Check

Open `data/manuscripts.json` in your repository. It should be two or three lines long and mention no paper titles at all. If it is thousands of lines, you replaced the wrong file — try again.

Checkpoint

All five files in `data/` are short and empty. The tracker now knows nothing, which is exactly right.

Part 3 — Tell the app who you are

Time: 10 minutes.

Four small edits. Each one follows the same pattern: open the file, click the pencil, change a line, commit. Take them slowly — a stray quotation mark is the most common thing that goes wrong here.

How to edit any file on GitHub Click the file → click the **pencil icon** (✎) at the top right → make your change → scroll down → **Commit changes** → **Commit changes** again.

Step 3.1 — `config/accounts.json` — which inboxes to watch

Open `config` → `accounts.json` and click the pencil.

Find the part that looks like this:

```
"accounts": [  
  {  
    "label": "Sathish Muthu",  
    "email": "drsathishmuthu@gmail.com",  
    "refreshTokenEnv": "GMAIL_REFRESH_TOKEN_SATHISH",  
    "provider": "gmail"  
  },  
  {  
    "label": "Dhibin Vikash Kolarpatti Ponnusamy",  
    "email": "dhibinvikash1@gmail.com",  
    "refreshTokenEnv": "GMAIL_REFRESH_TOKEN_DHIBIN",  
    "provider": "gmail"  
  }  
]
```

Change **only** the `label` and `email` lines to your own people:

```
"accounts": [  
  {  
    "label": "Dr Sarah Chen",  
    "email": "sarah.chen@gmail.com",  
    "refreshTokenEnv": "GMAIL_REFRESH_TOKEN_SATHISH",  
    "provider": "gmail"  
  },  
  {  
    "label": "Dr Raj Patel",  
    "email": "raj.patel.ortho@gmail.com",  
    "refreshTokenEnv": "GMAIL_REFRESH_TOKEN_DHIBIN",  
    "provider": "gmail"  
  }  
]
```

Leave `refreshTokenEnv` exactly as it is. `GMAIL_REFRESH_TOKEN_SATHISH` looks like a name but it is just the label on a storage box. Renaming it means changing it in a second file too, and forgetting the second one is a classic half-hour lost. Keep the odd names; they harm nothing.

Watching only one inbox? Delete the second block, including the comma that ends the first one:

```
"accounts": [  
  {  
    "label": "Dr Sarah Chen",  
    "email": "sarah.chen@gmail.com",  
    "refreshTokenEnv": "GMAIL_REFRESH_TOKEN_SATHISH",  
    "provider": "gmail"  
  }  
]
```

Watching three or more? Copy a block, change the label and email, and give it a new `refreshTokenEnv` name such as `GMAIL_REFRESH_TOKEN_THIRD`. You must then add that name to the workflow file as well — see Appendix C.

Step 3.2 — `assets/sync.js` — where your repository lives

Open `assets` → `sync.js` and click the pencil. Near line 29 you will find:

```
const OWNER = "ORG-Karur-DataCenter";  
const REPO = "Manuscript-Manager";
```

Change `OWNER` to **your GitHub username** (exactly as it appears in your repository's web address), and `REPO` to your repository name if you changed it:

```
const OWNER = "sarah-ortho-lab";  
const REPO = "Manuscript-Manager";
```

Keep the quotation marks and the semicolons. Change only the words between the quotes.

Step 3.3 — `config/notify.json` — where reminders should link to

Open `config` → `notify.json`. Find the last line before the closing brace:

```
"dashboardUrl": "https://org-karur-datacenter.github.io/Manuscript-Manager/#needs_action"
```

Replace it with your own address. It is built from your username and repository name, **all in lower case**:

```
"dashboardUrl": "https://sarah-ortho-lab.github.io/Manuscript-Manager/#needs_action"
```

GitHub website addresses are always lower case even when your username has capitals in it. Sarah-Ortho-Lab becomes sarah-ortho-lab.

While you are here, you may also want reminders for post-review revisions, not just pre-review amendments. Change:

```
"eventTypes": ["sent_back"],
```

to:

```
"eventTypes": ["sent_back", "revision_requested"],
```

Step 3.4 — assets/config.json — the Edit button

Open `assets` → `config.json`. It contains one line pointing at somebody else's service:

```
{
  "syncProxyUrl": "https://orgkarur-comms-sync.orgkarurdatacenter.workers.dev"
}
```

Empty it out for now:

```
{
  "syncProxyUrl": ""
}
```

Leaving the old address in would send your password to a stranger's service. Part 11 puts your own address here, if you decide you want the Edit button.

✓ Checkpoint

Four files edited and committed: `config/accounts.json`, `assets/sync.js`, `config/notify.json`, `assets/config.json`.

Part 4 — Let it read your email

Time: about 25 minutes. This is the longest part. Read it through once before starting.

Google will not let a program read a Gmail inbox just because you ask. You have to register the app with Google, then grant it permission from inside each inbox. What you get back is a long string of characters called a **refresh token** — a key that lets the app read that one inbox, and nothing else, until you take it away.

You will do Step 4.5 **once per inbox**.

Step 4.1 — Create a Google Cloud project

1. Go to <https://console.cloud.google.com/> and sign in with any Google account. It does not have to be one of the inboxes being watched.
2. At the top of the page there is a project dropdown (it may say *Select a project*). Click it → **New project**.
3. **Name:** Manuscript Manager . Leave the rest. Click **Create**.
4. Wait a few seconds, then make sure the dropdown at the top now shows *Manuscript Manager*. If it does not, click it and select your new project.

Everything for the rest of Part 4 must happen **inside this project**. If a screen looks unexpectedly empty, check the project name at the top first.

Step 4.2 — Switch on the Gmail API

1. In the search bar at the top, type **Gmail API** and click the result.
2. Click the blue **Enable** button.

Step 4.3 — Fill in the consent screen, and publish it

This is the step that most often goes wrong, in a way that does not show up for a week. Read the warning at the end before you click anything.

1. In the left menu, go to **APIs & Services → OAuth consent screen**. (On newer layouts it may sit under **Google Auth Platform → Branding**.)
2. Choose **External**, then **Create**.
3. Fill in the required fields — leave everything else blank, then click **Save and Continue**:
 - **App name:** Manuscript Manager
 - **User support email:** your own address
 - **Developer contact email:** your own address again

4. On the **Scopes** screen, click **Save and Continue** without adding anything.
5. On the **Test users** screen, click **Add users** and add **every Gmail address you want watched**. Click **Save and Continue**.
6. Go back to the **OAuth consent screen** summary page. Find the **Publishing status** box and click **PUBLISH APP**. Confirm. The status must now read **In production**.

⚠ Publish BEFORE you do Step 4.5, not after

While the app sits in *Testing*, Google expires every key it issues after **seven days**. Everything works perfectly for a week, and then the tracker stops, quietly, with an error nobody is watching for.

Publishing stops that — but only for keys made *after* you publish. Publishing does not rescue a key that was already issued. If you do these steps out of order, you must repeat Step 4.5 and replace the keys.

You will see a red "Google hasn't verified this app" warning once, in Step 4.5. That is normal and expected for an app used by its own author. You will be shown how to click past it.

Step 4.4 — Create the credentials

1. Left menu → **APIs & Services** → **Credentials**.
2. + **Create credentials** → **OAuth client ID**.
3. **Application type: Web application**.

It really must be *Web application*. *Desktop app* will not work with the browser method in the next step.

4. **Name:** Manuscript Manager .
5. Under **Authorised redirect URIs**, click + **Add URI** and paste exactly:

```
https://developers.google.com/oauthplayground
```

No trailing slash. Check it character by character.

6. Click **Create**.
7. A box appears with a **Client ID** and a **Client secret**. **Copy both into a notes file now**. You need them several times and the secret is awkward to find again.

They look roughly like:

```
Client ID:      1234567890-abcdefghijklmnopqrstuvwxyz.com
Client secret:  G0CSPX-AbCdEfGhIjKlMnOpQrSt
```

Step 4.5 — Get a key for an inbox

Do this once for each Gmail account in `config/accounts.json`.

Before you start, **sign out of every Google account in your browser except the one you are doing right now**, or use a private/incognito window. Having several accounts signed in is the second most common way this goes wrong: you authorise the wrong inbox and the tracker silently watches an empty mailbox.

1. Go to <https://developers.google.com/oauthplayground>.
2. Click the **gear icon (⚙)** at the top right.
3. Tick **Use your own OAuth credentials**.
4. Paste your **Client ID** and **Client secret** into the two boxes that appear. Close the gear panel.
5. On the left, under **Step 1 — Select & authorize APIs**, ignore the long list. At the very bottom is a box labelled **Input your own scopes**. Paste exactly:

```
https://www.googleapis.com/auth/gmail.readonly
```

6. Click the blue **Authorize APIs**.
7. Google asks you to sign in. **Choose the exact inbox you are making this key for**.
8. You will see **"Google hasn't verified this app"**. Click **Advanced**, then **Go to Manuscript Manager (unsafe)**. This is your own app; the warning only means Google has not reviewed it, which it does not need to for personal use.
9. Tick the box to allow reading email, then **Continue**.
10. You are returned to the playground, now on **Step 2**. Click **Exchange authorization code for tokens**.
11. A panel on the right fills with text. Find the line beginning **"refresh_token"** and copy the long value inside the quotation marks. It starts with `1//`.

```
"refresh_token": "1//0eXaMpLe-ThIsIsTheLongStringYouWant...",
```

12. **Paste it into your notes**, clearly labelled with which inbox it belongs to. You cannot see it again.

Now repeat from step 1 for the next inbox, in a fresh private window.

If no `refresh_token` appears, Google thinks this inbox has already granted permission. Go to <https://myaccount.google.com/permissions> while signed in as that inbox, find *Manuscript Manager*, remove its access, and repeat Step 4.5.

✓ Checkpoint

In your notes you have: one Client ID, one Client secret, and one refresh token per inbox — each labelled with its inbox.

Part 5 — Give it a free AI key

Time: 5 minutes.

Journal emails come in no standard format, so a small AI model reads each one and works out what it means. Both services below are free and neither asks for a card. Get **Groq** at minimum; **Gemini** as well is better, because when one runs out for the day the app falls back to the other.

Step 5.1 — Groq (the main one)

1. Go to <https://console.groq.com/keys>.
2. Sign in with Google or GitHub.
3. Click **Create API Key**, name it `Manuscript Manager`, click **Submit**.
4. Copy the key into your notes. It starts with `gsk_`. **You cannot see it again** — if you lose it, delete it and make another.

Step 5.2 — Google Gemini (the backup)

1. Go to <https://aistudio.google.com/apikey>.
2. Sign in with any Google account.
3. Click **Create API key** → choose your `Manuscript Manager` project if offered.
4. Copy the key into your notes. It starts with `AIza`.

A privacy note worth ten seconds. Whichever service you use reads the full text of your journal correspondence. Groq states it does not train on what you send. Google's free tier may use it to improve their products. If that matters to you or your co-authors, use Groq alone and skip Step 5.2 — the app works fine with one key.

Checkpoint

At least one AI key in your notes.

Part 6 — Put the keys into GitHub

Time: 10 minutes.

Now you hand everything you collected to GitHub, which will keep it hidden and use it every hour. Once saved, nobody — not even you — can read a secret back. You can only replace it. That is the point.

Step 6.1 — Open the secrets page

In your repository: **Settings** (along the top of the repository, not your account settings) → in the left menu, **Secrets and variables** → **Actions**.

You are on the **Secrets** tab. For each row of the table below:

1. Click **New repository secret**.
2. Type the **Name** exactly as written — capitals, underscores and all.
3. Paste the **Value**.
4. Click **Add secret**.

Step 6.2 — Add these

Name	Value
EMAIL_CLIENT_ID	the Client ID from Step 4.4
EMAIL_CLIENT_SECRET	the Client secret from Step 4.4
EMAIL_REFRESH_TOKEN_SATHISH	the refresh token for your first inbox
EMAIL_REFRESH_TOKEN_DHIBIN	the refresh token for your second inbox
GROQ_API_KEY	the Groq key from Step 5.1
GEMINI_API_KEY	the Gemini key from Step 5.2 (<i>skip if you did not make one</i>)

Yes, the names really are SATHISH and DHIBIN . They are the labels on the boxes, left over from the group this was built for, and they must match what you left in `config/accounts.json` in Step 3.1. The app does not read them as names. Watching only one inbox? Add only the first one.

Step 6.3 — Check your spelling

Your Secrets list should now show five or six entries. Compare each name against the table, letter by letter. A misspelled secret is invisible: GitHub accepts it happily and the app simply behaves as though you never added it.

Checkpoint

Five or six secrets listed, all spelled exactly as in the table.

Part 7 — Switch the website on

Time: 5 minutes.

1. In your repository: **Settings** → left menu, **Pages**.
2. Under **Build and deployment**:
 - **Source:** *Deploy from a branch*
 - **Branch:** `main`, and the folder `/ (root)`
3. Click **Save**.
4. Wait two or three minutes, then reload the page. A green banner appears with your website address:

```
https://<your-username>.github.io/Manuscript-Manager/
```

5. Open it. You will be asked for a password. Type `orgkarur2026` — the default, which you will change in Part 9.
6. You should see the dashboard, with a message saying "**No manuscripts yet**". That is correct. You have not synced anything.

If you get a 404 page, GitHub has not finished building yet. Wait five minutes and reload. If it persists after ten, check that the branch is `main` and the folder is `/ (root)`.

Write this address down. It is the link you will share with your co-authors, and it is also what you typed into `config/notify.json` in Step 3.3 — this is a good moment to check those two match.

✓ Checkpoint

The website opens, accepts the default password, and says "No manuscripts yet".

Part 8 — Run the first sync

Time: 10 minutes, mostly waiting.

Step 8.1 — Start it by hand

- 1. In your repository, click the **Actions** tab along the top.
- 2. If you see a banner saying workflows are disabled, click **I understand my workflows, go ahead and enable them**.
- 3. In the left list, click **Sync manuscript tracker**.
- 4. On the right, click the **Run workflow** dropdown.
- 5. Leave every box empty and click the green **Run workflow**.

Wait about fifteen seconds and reload the page. A run appears with a spinning yellow dot.

Step 8.2 — Watch what happens

Click the run, then click the **sync** box to see the log unfold. The first run looks back 30 days, so it has a lot to read and takes **five to fifteen minutes**.

You are looking for lines like:

```
Fetching messages for sarah.chen@gmail.com ...
prefilter: 214 excluded without an AI call
classified 38 messages
filed: Outcomes of Robotic-Assisted Total Knee Arthroplasty – new_submission
```

When the dot turns into a **green tick**, it worked.

Step 8.3 — Look at your dashboard

Open your website address again and reload the page (**Ctrl + Shift + R**, or **Cmd + Shift + R** on a Mac, to make sure you get the fresh version).

Your papers should be there, sorted into sections:

Section	What is in it
With the journal	Submitted, under review, or accepted and awaiting publication.
Needs action	Rejected and needing a new home, or sent back before review.
Revisions pending	Peer review is done and the journal wants changes from you.
Published	Out, with a DOI.

From now on this happens by itself, every hour. You never need to press anything again.

Step 8.4 — If the run failed (red X)

Click into the failed run and read the last few red lines. The usual causes:

What the log says	What it means	Fix
<code>invalid_grant</code>	The refresh token is wrong, or was made before you published the consent screen	Redo Step 4.3 (publish), then Step 4.5, then replace the secret
<code>no refresh token for ...</code>	A secret name does not match <code>config/accounts.json</code>	Compare Step 3.1 and Step 6.2 letter by letter
<code>all providers failed</code>	No AI key, or a wrong one	Check <code>GR0Q_API_KEY</code> in Step 6.2
<code>Gmail API has not been used</code>	Step 4.2 was missed	Enable the Gmail API, wait two minutes, run again
Nothing filed, no error	It worked; there was simply no journal mail in the last 30 days	Try a wider sweep — see Appendix B

Checkpoint

A green tick in Actions, and papers on your dashboard.

Part 9 — Change the password

Time: 5 minutes. Do not skip this.

Everyone who has read this guide knows the default password. Change it now.

1. Go to `https://<your-username>.github.io/Manuscript-Manager/tools/password.html` (your website address with `tools/password.html` on the end).
2. Type the password you want into the box. Nothing is sent anywhere — the scrambling happens inside your own browser tab.
3. A line of text appears underneath, looking like:

```
const PASSWORD_HASH = "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08";
```

4. Click **Copy this line**.
5. In your repository, open `assets` → `auth.js`, click the pencil, and find line 23 — the one that begins `const PASSWORD_HASH =`.
6. Select that whole line and paste over it.
7. **Commit changes**.
8. Wait two minutes, then open your dashboard in a private window. Your new password should work and `orgkarur2026` should not.

Tell your co-authors the new password directly — in person, or by a message, not by putting it in the repository. Anything committed to a public repository is public.

✓ Checkpoint

The old password is refused; the new one works.

Part 10 — WhatsApp reminders

Time: 10 minutes.

When a journal sends a paper back with a two-week deadline, the tracker can message your phone: when it first sees it, again at three days out, again at one day, and once more if the date passes.

This uses **CallMeBot**, a free service that needs no account and no card. Each person who wants reminders does Step 10.1 on their own phone.

Reminders also arrive as **GitHub issues** whether or not you set this up. Those need nothing at all, and GitHub's phone app will notify you. If you would rather not use a third-party WhatsApp service, skip this part — you are still covered.

Step 10.1 — Each person activates the bot

Do this on the phone that should receive reminders.

1. First, check the current number at <https://www.callmebot.com/blog/free-api-whatsapp-messages/>.

This matters. The number has changed before, and old ones get reassigned to real people — messaging a stale number sends your deadlines to a stranger. Take the number from that page, not from this guide.

2. Save that number to your contacts under any name.
3. Send it exactly this message on WhatsApp:

I allow callmebot to send me messages

The word `callmebot` must be exactly that, whatever you named the contact.

4. Within a couple of minutes it replies: *"API Activated for your phone number. Your APIKEY is 123123"*
5. Write down that **API key** and the **phone number in full international form without the plus** — for example `919600856806` for an Indian number.

Nothing came back? CallMeBot is one person's side project and does sometimes not answer. Check the number and the exact wording first. Their own advice is to wait 24 hours and try once more, rather than repeating it every few minutes.

Step 10.2 — Add the recipients as a secret

Back in your repository: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

- **Name:** `WHATSAPP_RECIPIENTS`
- **Value:** one line per person, in this exact shape:

```
[{"name":"Dr Sarah Chen","phone":"919600856806","apiKey":"123456"},  
{"name":"Dr Raj Patel","phone":"918778138148","apiKey":"654321"}]
```

Watch the punctuation: square brackets around the whole thing, curly braces around each person, a comma between people, no comma after the last one.

Phone numbers go in the secret and **never** into a file in the repository. A published phone number cannot be revoked the way a key can.

Step 10.3 — Test it

1. **Actions** tab → **Test WhatsApp reminders** → **Run workflow**.
2. There are **two** dropdowns. Set both:
 - **What to do:** `send-test`
 - **Which service to send through:** `callmebot`
3. Run it. Everyone on the list should get a short message within a minute.

⚠ The second dropdown is set to `meta` by default

Change it to `callmebot` or the test will fail, even though everything is set up correctly. `meta` is a different WhatsApp service that you have not configured.

This catches people out because the *real* hourly reminders do use CallMeBot — only this test button defaults to the other one. So it is entirely possible for the test to fail while your actual reminders are arriving perfectly. If the test fails, check this dropdown before changing anything else.

If nothing arrives, open the run's log. It names each recipient and whether a key was found, showing only the last four digits of each number — because these logs are public on a public repository.

✅ Checkpoint

A test message arrived on each phone.

Part 11 — The Edit button (optional)

Time: 25 minutes. Everything above works without this.

Should you bother?

Without this part, the dashboard is read-only. You can see everything, but you cannot correct a title the AI got slightly wrong, move a paper to a different section, merge two cards that are really the same paper, or press **Sync now**.

The reason it is extra work is worth understanding. The dashboard is just a page of files — there is no server behind it. To change anything, something has to hold a key with permission to write to your repository, and that key cannot live in the page itself, because the page is public. So you run a tiny free service, at Cloudflare, that holds the key for you.

It is free, it takes about 25 minutes, and you can do it any time later.

Step 11.1 — Make a GitHub key for it

1. Go to <https://github.com/settings/personal-access-tokens/new>.
2. **Token name:** `Manuscript Manager worker`
3. **Expiration:** choose the longest offered. Note the date — you will have to repeat this when it expires, and the app stops working the day it does.
4. **Repository access:** *Only select repositories* → choose your `Manuscript-Manager`.
5. **Permissions** → **Repository permissions**, set exactly two:
 - **Actions:** *Read and write*
 - **Contents:** *Read and write*

Nothing else. A key limited like this cannot read your secrets, cannot touch another repository, and cannot do anything outside this app.

6. **Generate token**, then copy it into your notes. It starts with `github_pat_`. You cannot see it again.

Step 11.2 — Edit the worker file first

The service is one file in your repository, and it also needs to know your username.

1. Open `worker` → `src` → `index.js` and click the pencil.
2. Near line 28, exactly as you did in Step 3.2, change:

```
const OWNER = "ORG-Karur-DataCenter";  
const REPO = "Manuscript-Manager";
```

to your own username and repository name.

3. **Commit changes.**

- Now open the file again (not the editor — just the file view) and click the **Copy raw file** button at the top right of the code. The whole file is now on your clipboard. Leave this tab open.

Step 11.3 — Create the Cloudflare service

- Go to <https://dash.cloudflare.com/sign-up> and make a free account. Verify your email.
- In the left menu, click **Workers & Pages** → **Create** → **Start with Hello World!** → **Get started**.
- Name:** `manuscript-sync`. Click **Deploy**.
- When it finishes, click **Edit code** (or **Continue to project** → **Edit code**).
- In the code editor, select everything (**Ctrl/Cmd + A**) and paste your file over it (**Ctrl/Cmd + V**).
- Click **Deploy** at the top right.

Step 11.4 — Give it the three settings

Still in Cloudflare, leave the code editor and go to your worker's **Settings** → **Variables and Secrets**.

Add these three:

Name	Type	Value
<code>GITHUB_TOKEN</code>	Secret	the <code>github_pat_...</code> key from Step 11.1
<code>APP_PASSWORD</code>	Secret	your dashboard password from Part 9
<code>ALLOWED_ORIGIN</code>	Text	<code>https://<your-username>.github.io</code>

`ALLOWED_ORIGIN` is just the first part of your website address — the bit ending in `.github.io`, with **no** repository name and **no** slash at the end. It stops any other website calling your service from a visitor's browser.

Click **Deploy** (or **Save and deploy**) after adding them.

Step 11.5 — Check it is alive

- Find your worker's address on its overview page. It looks like `https://manuscript-sync.something.workers.dev`.
- Open that address in your browser with `/health` on the end:

```
https://manuscript-sync.something.workers.dev/health
```

3. You should see something like this (there will be a little more after it, listing the settings it can see — that is normal):

```
{"ok":true,"configured":true,"missing":[],"methods":["GET","POST","PATCH","DELETE","OPTIONS"]}
```

- **configured: true** — both settings arrived. Good.
 - **configured: false** — the **missing** list names exactly which one is absent or misspelled. Fix it in Step 11.4 and reload.

Step 11.6 — Point the dashboard at it

1. In your repository, open **assets** → **config.json** and click the pencil.
2. Put your worker address between the quotes — **no slash at the end**:

```
{  
  "syncProxyUrl": "https://manuscript-sync.something.workers.dev"  
}
```

3. **Commit changes.**
4. Wait two minutes, reload your dashboard, and open any paper. An **Edit** button now appears, and so does **Sync now** in the header.

✓ Checkpoint

`/health` says `configured: true`, and the Edit button appears on the dashboard.

Using it day to day

What you will actually do

Almost nothing. It runs every hour. Open the dashboard when you want to know where something stands, and answer a WhatsApp message when a deadline is near.

Correcting something

Open a paper and press **Edit**. You can fix the title, the journal, the manuscript number, the DOI, or move it to a different section, and add notes for anything the emails never said.

What you type by hand stays. This matters more than it sounds. Every field is worked out again from email, every hour — so a correction that was merely written down would be silently undone by the next letter from the journal. The app instead *pins* what you set: the hourly sync is forbidden from touching it, and the field is marked **set by hand** wherever it appears.

Each pinned field has a **use automatic again** link when you want to hand it back.

Two cards that are really one paper

It happens — the same paper submitted to two journals under slightly different titles. Open one card, press **Edit**, and use **Merge into another manuscript**. Every card is offered, with likely matches at the top.

Seeing everything by journal

The **By journal** view shows one card per journal with the papers you have there, in date order. Useful before you submit somewhere new.

Deadlines that are guesses

Where a journal states a deadline, the tracker uses it. Where it does not, it falls back to that journal's usual window from `config/deadlines.json` and marks the date **estimated** everywhere it appears — on the card, in the drawer, and in the WhatsApp message. A guessed date that looked certain would be worse than no date, so it always says which it is.

Correct one with **Edit** and the reminders follow your date instead.

When something goes wrong

What you see	What it usually is	What to do
Dashboard says "No manuscripts yet" after a green sync	There was genuinely no journal mail in the window	Run a wider sweep — Appendix B
Everything stopped after about a week	The consent screen was published <i>after</i> the keys were made	Redo Step 4.5 and replace the token secrets
A red X in Actions	Open the run and read the last red lines	See the table in Step 8.4
Changes to a file do not show on the site	The browser cached the old version	Reload with Ctrl/Cmd + Shift + R
No Edit button	Part 11 not done, or <code>assets/config.json</code> is empty or wrong	Check Step 11.6, then <code>/health</code>
Edit says "not authorised"	<code>APP_PASSWORD</code> in Cloudflare does not match the dashboard password	Reset it in Step 11.4
Edit fails mentioning Contents	The GitHub key has only <i>Actions</i> permission	Add Contents: Read and write in Step 11.1
No WhatsApp messages	CallMeBot key wrong, or the number changed	Redo Step 10.1, checking the number on their page
Papers appear that are not yours	Part 2 was skipped	Do Part 2 now, then run a sync
A paper is in the wrong section	The AI read a letter differently from you	Open it, Edit , move it. Your choice is then pinned

Where to look first, always

Actions tab → the most recent run → click into it. The app is written to explain itself in that log: it names the file, the setting, or the person involved rather than reporting that something went wrong. Nine problems in ten are named there in plain words.

Appendix A — Everything you set, in one place

GitHub secrets

(Settings → Secrets and variables → Actions)

Name	Required?	Where it came from
EMAIL_CLIENT_ID	Yes	Step 4.4
EMAIL_CLIENT_SECRET	Yes	Step 4.4
EMAIL_REFRESH_TOKEN_SATHISH	Yes	Step 4.5, first inbox
EMAIL_REFRESH_TOKEN_DHIBIN	If a second inbox	Step 4.5, second inbox
GROQ_API_KEY	At least one AI key	Step 5.1
GEMINI_API_KEY	Optional backup	Step 5.2
WHATSAPP_RECIPIENTS	Optional	Step 10.2
GCHAT_WEBHOOK_URL	Optional	Appendix C

Files you edited

File	What changed	Step
data/ × 5	Emptied out	Part 2
config/accounts.json	Your names and emails	3.1
assets/sync.js	Your GitHub username	3.2
config/notify.json	Your website address	3.3
assets/config.json	Emptied, then your worker address	3.4, 11.6
assets/auth.js	Your password	Part 9
worker/src/index.js	Your GitHub username	11.2

Cloudflare settings

(only if you did Part 11)

Name	Type
GITHUB_TOKEN	Secret
APP_PASSWORD	Secret
ALLOWED_ORIGIN	Text

Appendix B — Filling in older papers

The hourly sync only looks at mail newer than the last time it ran, which is right for keeping up and useless for going back. The first run reaches back 30 days; to go further, sweep an explicit window.

1. **Actions** → **Sync manuscript tracker** → **Run workflow**.

2. Fill in:

- **Sweep from:** 2025-01-01 (a date, in that exact format)
- **Sweep to:** 2025-06-30 , or leave blank for "up to now"
- **Messages to fetch per mailbox:** 400

3. Run it.

Sweep a few months at a time rather than three years at once. The free AI tiers have daily limits, and anything over the limit is **deferred, not lost** — but a run that defers most of its work has little to show for itself. Run it again the next day to continue.

A sweep deliberately does not move the "caught up to here" marker, so it cannot make the tracker skip the gap between the sweep and today.

Appendix C — Optional extras

More than two inboxes

After adding a third block to `config/accounts.json` (Step 3.1) with a new `refreshTokenEnv` such as `EMAIL_REFRESH_TOKEN_THIRD`, you must also tell the workflow about it.

Open `.github` → `workflows` → `sync-manuscripts.yml`, find the list that reads:

```
GMAIL_REFRESH_TOKEN_SATHISH: ${ secrets.GMAIL_REFRESH_TOKEN_SATHISH }
GMAIL_REFRESH_TOKEN_DHIBIN: ${ secrets.GMAIL_REFRESH_TOKEN_DHIBIN }
```

and add a matching line underneath, keeping the indentation identical:

```
GMAIL_REFRESH_TOKEN_THIRD: ${ secrets.GMAIL_REFRESH_TOKEN_THIRD }
```

Then add the secret itself, exactly as in Step 6.2.

Google Chat instead of, or as well as, WhatsApp

The simplest channel of the three, and the only one where everyone sees the same thread.

1. In Google Chat, create a space for this.
2. **Space name** → **Apps & integrations** → **Webhooks** → **Add webhooks**.
3. Name it, then **Copy** the address.
4. Add it as the GitHub secret `GCHAT_WEBHOOK_URL`.

That is all. Each paper posts under its own thread.

The address is a credential — anyone holding it can post into that space. Keep it in the secret, never in a file.

An Outlook mailbox

The tracker reads Outlook as well as Gmail, through a Microsoft app registration. It is free but noticeably more involved than Gmail. The full walkthrough is in the main `README.md` under *Adding an Outlook mailbox*.

Meta's official WhatsApp service

CallMeBot is one person's side project. Meta's own WhatsApp Cloud API is the robust alternative and is also free at this volume, but setting it up takes most of an evening: a developer app, a system-user token, a message template submitted for approval, and each recipient verified with a code. The full walkthrough is in `README.md` under *Meta's WhatsApp Cloud API*.

Only go there if CallMeBot has let you down.

Turning off the GitHub issue reminders

Set `"githubIssues": false` in `config/notify.json`.

Appendix D — Keeping it healthy

Three things expire or drift. None is urgent, all are easy.

What	When	What happens if you forget
The GitHub key for the worker (Step 11.1)	On the date you chose	Sync now and Edit stop working. The hourly sync is unaffected.
Gmail refresh tokens	Only if you change your Google password, revoke access, or the consent screen slips back to <i>Testing</i>	The sync fails with <code>invalid_grant</code>
The CallMeBot number	Whenever they change it	Reminders stop, or reach a stranger

Once a month, glance at the Actions tab. A column of green ticks means everything is fine. A run of red ones means something expired, and the log will say which.

A last word

This app was built for one research group, over months, mostly by fixing things that went wrong in ways nobody predicted. Where the instructions above seem oddly insistent — publish the consent screen *first*, check that `.github` uploaded, do not skip Part 2 — it is because each of those cost somebody a long evening.

If a step does not match what you see on screen, these services redesign their pages often. The words on the buttons change; what you are looking for does not. Read the step's explanation of *what* you are trying to achieve, and look for the button that does that.